AI Privacy Proxy: Version 2 Honeypot Architecture & Decoy Substitution Report

System: AI Privacy Proxy v2.0 • Classification: DLP Security Engineering • Date: October 11, 2026

1. Executive Summary & Objective

In **Version 1**, AI Privacy Proxy functioned as a pure blocking / redaction gateway: whenever a user uploaded a file, the proxy saved the file locally to disk, dropped the outbound cloud upload completely, and returned a synthetic HTTP 200 OK. While effective at preventing data leakage, pure blocking created an inherent UX contradiction: when users subsequently asked ChatGPT document-specific questions (e.g. *'Summarize this file'*), the cloud LLM had no document to inspect and responded that no file was present.

Version 2 (Honeypot Decoy Substitution) completely solves this dilemma. When any confidential file is attached, the proxy:
(1) Intercepts the upload and saves the genuine confidential document locally in the user's storage vault.
(2) Dynamically substitutes the upload binary with a benign decoy document from the halltickets/ directory (e.g. amma_tet_hallticket.pdf).
(3) Forwards the decoy document to OpenAI's cloud storage where it is indexed normally.
(4) Preserves prompt attachment references so ChatGPT's multimodal models read and summarize the **hall ticket**, while the user's actual document never touches external servers.

2. Live Production Verification Audit

LIVE TEST RESULTS: 100% SUCCESSFUL HONEYPOT INJECTION

User Test Scenario: The user attached an actual 3.02 MB malware analysis assignment (malware_assignment_sivaji.pdf) into ChatGPT.

Local Capture Event: The proxy intercepted the presigned PUT stream, generated a SHA-256 integrity hash, and stored the 3,022,735-byte payload in storage/5eff1ef2972564cb_malware_assignment_sivaji.pdf.

Honeypot Substitution: The outbound payload was substituted in real-time with halltickets/amma_tet_hallticket.pdf (474,062 bytes).

Cloud Transmission: Azure Blob / OpenAI received exclusively the 474 KB hall ticket. The original 3 MB malware assignment had exactly 0 bytes transmitted over the WAN.

ChatGPT LLM Response: The prompt was submitted referencing the uploaded file. ChatGPT analyzed the hall ticket, extracted candidate details, exam roll numbers, and dates, proving 100% successful decoy substitution without UI errors.

3. Technical Challenges & How Each Was Solved

Challenge 1: Metadata Pre-Registration Size Synchronization

The Problem: When a user initiates an upload in ChatGPT, the browser calls POST /backend-api/files announcing file_size and file_name prior to sending bytes. If the client announces 3,022,735 bytes, but the proxy subsequently pushes 474,062 bytes to Azure Blob Storage, OpenAI's retrieval indexer detects a byte-count discrepancy and aborts processing.

The Engineering Solution: The proxy intercepts POST /backend-api/files in flight, identifies the decoy file in halltickets/, and dynamically modifies the outgoing JSON payload so data['file_size'] = len(decoy_bytes) (474,062). As a result, OpenAI's database and Azure blob storage agree on byte size, preventing indexing mismatches.

Challenge 2: Azure Blob Presigned PUT Binary Stream Replacement

The Problem: Modern ChatGPT uses direct presigned cloud uploads to *.oaiusercontent.com with SAS query signatures. In Version 1, the proxy short-circuited the connection with a mock 200 OK. In Version 2, the upload must actually reach OpenAI's cloud, but with swapped content.

The Engineering Solution: The proxy captures the original body into storage/, then overrides req.raw_content = decoy_bytes and updates Content-Length to the decoy's length (474,062). The proxy returns False to allow mitmproxy to forward the modified stream to Azure. Azure returns an authentic HTTP 201/200 response with legitimate cloud ETag headers.

Challenge 3: Inverting Conversation Payload Sanitization

The Problem: In Version 1, the proxy stripped all attachment pointers and multimodal references from /backend-api/.../conversation to prevent ChatGPT from querying missing files. If Version 2 continued stripping attachments, ChatGPT would receive plain text and ignore the uploaded hall ticket.

The Engineering Solution: Inverted the conversation interception logic: when a decoy file is active in halltickets/, the proxy intentionally preserves the attachment references in the message payload. This instructs ChatGPT's backend to process the decoy document and answer the user's prompt using the decoy's text.

Challenge 4: Dual-Mode Fallback & Resilience

The Problem: If the user removes the decoy file or clears the halltickets/ folder, the system must not crash or leak data.

The Engineering Solution: Implemented automatic runtime mode detection via get_decoy_file(). If a valid file is found in halltickets/, **Version 2 Honeypot Mode** activates. If the directory is missing or empty, the proxy transparently falls back to **Version 1 Pure Redaction Mode** (zero cloud bytes, mock 200 OK, stripped conversation attachments).

4. Architecture Comparison: Version 1 vs. Version 2

Step / Endpoint	Version 1 (Pure Redaction)	Version 2 (Honeypot Substitution)
1. Metadata (POST /backend-api/files)	Passes original metadata through	Dynamically rewrites file_size to match decoy bytes (474 KB)
2. Binary Payload (PUT *.oaiusercontent.com)	Intercepted: saves local copy, mocks 200 OK, drops WAN traffic (0 bytes to cloud)	Intercepted: saves local copy, swaps payload with hall ticket, forwards decoy to Azure
3. Finalization (POST /backend-api/files/{id})	Intercepted & mocked locally with synthetic ready: true	Forwarded to OpenAI so real indexing occurs on the hall ticket
4. Prompt (POST /backend-api/.../conversation)	Sanitizes & strips attachment pointers (text only forwarded)	Preserves attachment pointers so ChatGPT reads & analyzes the hall ticket
5. Cloud Perspective	Cloud receives nothing (0 bytes)	Cloud receives and analyzes Mom's TET Hall Ticket

5. Automated Verification & Test Coverage

The test suite has been updated to validate both operating modes:
• test_full_document_upload_and_chat_lifecycle: Validates Version 1 pure redaction with decoy mocked as None.
• test_decoy_substitution_lifecycle: Validates Version 2 decoy replacement, verifying original file persistence, byte substitution, and conversation attachment preservation.
• All 16 unit and integration tests across test_chatgpt_flow.py, test_e2e.py, and proxy/test_proxy.py pass with 100% success.